

prac 3 Apply filtering, sorting, aggregation, and GroupBy operations on real-world datasets.

```
# Import Pandas library
```

```
import pandas as pd
```

```
# Create a Pandas DataFrame from employee data and display it
```

```
data = {"Emp_ID" : [101,102,103,104,105,106,107,108],
        "NAME": ["Rahul","Priya","Amit","Riya","Karan","Neha","Vivek","Pooja"],
        "Department": ["IT","HR","IT","FINANCE","HR","FINANCE","IT","HR"],
        "City": ["Surat","Navsari","Surat","Valsad","Surat","Navsari","Valsad","Surat"],
        "Salary": [45000,38000,52000,48000,40000,51000,60000,42000],
        "Experience": [2,1,5,4,2,6,8,3]}
df = pd.DataFrame(data)
print(df)
```

	Emp_ID	NAME	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
1	102	Priya	HR	Navsari	38000	1
2	103	Amit	IT	Surat	52000	5
3	104	Riya	FINANCE	Valsad	48000	4
4	105	Karan	HR	Surat	40000	2
5	106	Neha	FINANCE	Navsari	51000	6
6	107	Vivek	IT	Valsad	60000	8
7	108	Pooja	HR	Surat	42000	3

Step 2 : Filter Data

```
# Filter the DataFrame to display only employees from the IT department
```

```
df[df["Department"] == "IT"]
```

	Emp_ID	NAME	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
2	103	Amit	IT	Surat	52000	5
6	107	Vivek	IT	Valsad	60000	8

Step 3 : Display employees with salary greater than 50000

```
# Filter the DataFrame to display employees with a salary greater than 50,000
```

```
df[df["Salary"] > 50000]
```

	Emp_ID	NAME	Department	City	Salary	Experience
2	103	Amit	IT	Surat	52000	5
5	106	Neha	FINANCE	Navsari	51000	6
6	107	Vivek	IT	Valsad	60000	8

Display employees from surat

```
# Filter the DataFrame to display employees live in Surat
```

```
df[df["City"] == "Surat"]
```

	Emp_ID	NAME	Department	City	Salary	Experience
0	101	Rahul	IT	Surat	45000	2
2	103	Amit	IT	Surat	52000	5
4	105	Karan	HR	Surat	40000	2
7	108	Pooja	HR	Surat	42000	3

Apply Multiple Conditions

```
# Filter employees who are in the IT department and have a salary greater than 50,000
```

```
df[(df["Department"] == "IT") & (df["Salary"] > 50000)]
```

	Emp_ID	NAME	Department	City	Salary	Experience	
2	103	Amit	IT	Surat	52000	5	
6	107	Vivek	IT	Valsad	60000	8	

sort Salary in ascending order

```
# Sort the DataFrame by Salary in ascending order
```

```
df.sort_values(by="Salary")
```

	Emp_ID	NAME	Department	City	Salary	Experience	
1	102	Priya	HR	Navsari	38000	1	
4	105	Karan	HR	Surat	40000	2	
7	108	Pooja	HR	Surat	42000	3	
0	101	Rahul	IT	Surat	45000	2	
3	104	Riya	FINANCE	Valsad	48000	4	
5	106	Neha	FINANCE	Navsari	51000	6	
2	103	Amit	IT	Surat	52000	5	
6	107	Vivek	IT	Valsad	60000	8	

sort salary in descending

```
# Sort the DataFrame by Salary in descending order
```

```
df.sort_values(by="Salary", ascending=False)
```

	Emp_ID	NAME	Department	City	Salary	Experience	
6	107	Vivek	IT	Valsad	60000	8	
2	103	Amit	IT	Surat	52000	5	
5	106	Neha	FINANCE	Navsari	51000	6	
3	104	Riya	FINANCE	Valsad	48000	4	
0	101	Rahul	IT	Surat	45000	2	
7	108	Pooja	HR	Surat	42000	3	
4	105	Karan	HR	Surat	40000	2	
1	102	Priya	HR	Navsari	38000	1	

Sort By Department And Salary

```
# Sort the DataFrame by Department in ascending order and Salary in descending order
```

```
df.sort_values(by=["Department", "Salary"], ascending=[True, False])
```

	Emp_ID	NAME	Department	City	Salary	Experience	
5	106	Neha	FINANCE	Navsari	51000	6	
3	104	Riya	FINANCE	Valsad	48000	4	
7	108	Pooja	HR	Surat	42000	3	
4	105	Karan	HR	Surat	40000	2	
1	102	Priya	HR	Navsari	38000	1	
6	107	Vivek	IT	Valsad	60000	8	
2	103	Amit	IT	Surat	52000	5	
0	101	Rahul	IT	Surat	45000	2	

Step 4 : perform Aggregation

```
# Calculate the total salary of all employees
```

```
df["Salary"].sum()
```

```
np.int64(376000)
```

```
# Find the highest salary from all employees
```

```
df["Salary"].max()
```

```
60000
```

```
# Calculate the average salary of all employees
```

```
df["Salary"].mean()
```

```
np.float64(47000.0)
```

```
# Calculate the median of salary all employees
```

```
df["Salary"].median()
```

```
46500.0
```

```
# Count the number of non-missing salary values
```

```
df["Salary"].count()
```

```
np.int64(8)
```

Step 5 : perform GroupBy Operations

```
# Calculate the average salary of each department
```

```
df.groupby("Department")["Salary"].mean()
```

Salary	
Department	
FINANCE	49500.000000
HR	40000.000000
IT	52333.333333

```
dtype: float64
```

```
# Find the minimum salary in each department
```

```
df.groupby("Department")["Salary"].min()
```

Salary	
Department	
FINANCE	48000
HR	38000
IT	45000

```
dtype: int64
```

```
# Count the number of employees in each department based on their salary
```

```
df.groupby("Department")["Salary"].count()
```

Salary	
Department	
FINANCE	2
HR	3
IT	3

```
dtype: int64
```

```
# Find the highest salary in each department
```

```
df.groupby("Department")["Salary"].max()
```

	Salary
Department	
FINANCE	51000
HR	42000
IT	60000

dtype: int64

```
# Calculate the total salary for each department
```

```
df.groupby("Department")["Salary"].sum()
```

	Salary
Department	
FINANCE	99000
HR	120000
IT	157000

dtype: int64

Citywise Average Salary

```
# Calculate the average salary for each city
```

```
df.groupby("City")["Salary"].mean()
```

	Salary
City	
Navsari	44500.0
Surat	44750.0
Valsad	54000.0

dtype: float64

Step 6: Multiple Aggregation Functions

```
# Calculate the mean, minimum, maximum, and count of salary for each department
```

```
df.groupby("Department")["Salary"].agg(["mean", "min", "max", "count"])
```

	mean	min	max	count	
Department					
FINANCE	49500.000000	48000	51000	2	
HR	40000.000000	38000	42000	3	
IT	52333.333333	45000	60000	3	

Step 7 : Save the Result

```
df.to_csv("Emp")
```

